UNIVERSITY TEXTBOOK SERIES

THE COMPLETE AI & MACHINE LEARNING ROADMAP

From Absolute Beginner to Advanced Practitioner

A comprehensive mathematical, algorithmic, and practical guide for B.Tech Artificial Intelligence & Data Science students, engineers, and researchers.

Primary Target Audience: B.Tech AI & DS, Aspiring AI Engineers
Prerequisites: Basic High-School Mathematics

Edition: 2026 Academic Edition
Focus: Theory, Code, Intuition & Visuals

Table of Contents & Visual Roadmap

A structured 14-module progression from foundational mathematics to state-of-the-art Generative AI.

Part I: Core Foundations

- Module 1: Math & Python Foundations
- Module 2: Artificial Intelligence Foundations
- Module 3: Data Preparation & EDA

Part II: Classical Machine Learning

- Module 4: Supervised Machine Learning
- Module 5: Unsupervised Machine Learning
- Module 6: Reinforcement Learning

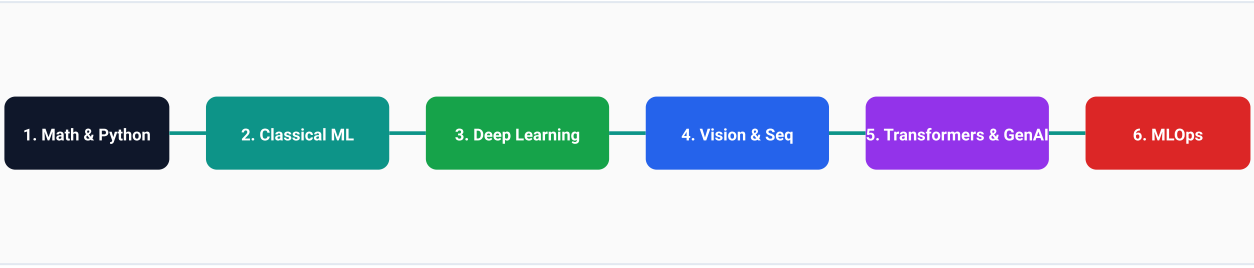
Part III: Deep Learning Architectures

- Module 7: Artificial Neural Networks
- Module 8: Deep Learning Practice
- Module 9: CNN & Computer Vision
- Module 10: RNN, LSTM & GRU

Part IV: Frontiers & MLOps

- Module 11: Attention & Transformers
- Module 12: Generative AI & LLMs
- Module 13: Specialized AI Domains
- Module 14: Deployment & MLOps

Visual Learning Roadmap



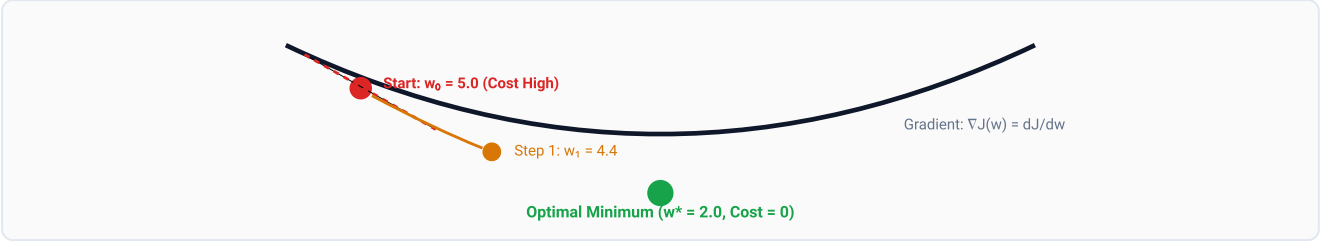
MODULE 1

Mathematical & Python Foundations

1.1 Learning Objectives & Core Intuition

Master the mathematical building blocks of machine learning: Linear Algebra (vectors/matrices), Differential Calculus (gradient descent), and Probability. Optimization algorithms rely on calculus to find parameter configurations that minimize error.

1.2 Concept Visualization: Gradient Descent Surface



1.3 Optimization Equations & Numerical Example

Given cost function $J(w) = w^2 - 4w + 4$, calculate the step update from $w^{(0)} = 5$ with learning rate $\alpha = 0.1$:

Gradient Equation: dJ/dw = 2w − 4 | Update: $w^{(t+1)} = w^{(t)} - \alpha \cdot (dJ/dw)$

- Step 1 Gradient at w=5: $2(5) - 4 = 6.0$
- Updated Weight: $w^{(1)} = 5.0 - (0.1 \times 6.0) = 4.40$

1.4 Python Code Implementation

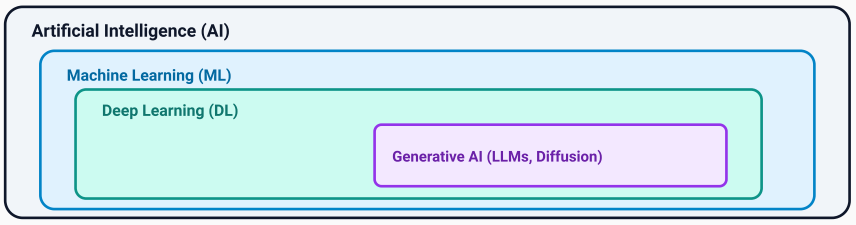
```
import numpy as np def gradient_descent(w_start, alpha, iterations): w = w_start for i in ran
ge(iterations): grad = 2 * w - 4 # Derivative of J(w) = w^2 - 4w + 4 w = w - alpha * grad print(f"I
ter {i+1}: w = {w:.4f}, Cost = {(w**2 - 4*w + 4):.4f}") return w gradient_descent(5.0, 0.1, 5)
```

MODULE 2

Artificial Intelligence Foundations

2.1 AI vs. Machine Learning vs. Deep Learning vs. GenAI

Artificial Intelligence encompasses the broader science of creating smart systems. Machine Learning relies on statistical pattern recognition, Deep Learning uses multi-layered neural networks, and Generative AI produces synthetic media from learned representations.



2.2 Paradigm Comparison Table

Paradigm	Input Data	Core Mechanism	Primary Output
Symbolic AI	Rules & Facts	Inference Engines & Logic Trees	Deterministic Decisions
Classical ML	Structured Features	Statistical Optimization (SVM, Trees)	Predictions / Labels
Deep Learning	Raw Data (Pixels, Audio)	Hierarchical Neural Feature Learning	High-level Representations
Generative AI	Prompts / Sequences	Transformer Attention / Diffusion	Synthetic Text / Images

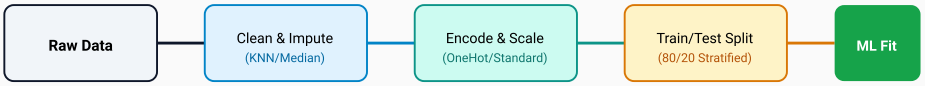
MODULE 3

Data Preparation & Exploratory Data Analysis

3.1 Data Pipeline Workflow

Raw real-world data contains missing values, noisy features, and scale discrepancies. Proper preprocessing prevents data leakage and ensures models converge efficiently during training.

3.2 Preprocessing Workflow Visual



3.3 Scaler Equations

StandardScaler (Z-Score): $z = (x - \mu) / \sigma$ | MinMaxScaler: $x' = (x - x_{min}) / (x_{max} - x_{min})$

3.4 Preprocessing Code Example

```
import pandas as pd from sklearn.preprocessing import StandardScaler, OneHotEncoder from skle
arn.compose import ColumnTransformer # Sample tabular dataset df = pd.DataFrame({'Age': [25, 45, 3
5], 'City': ['NY', 'SF', 'NY'], 'Income': [50000, 120000, 85000]}) # Column Transformer for scaling
numericals & encoding categoricals ct = ColumnTransformer(transformers=[ ('num', StandardScaler(),
['Age', 'Income']), ('cat', OneHotEncoder(), ['City']) ]) processed_data = ct.fit_transform(df) pri
nt("Processed Feature Array Shape:", processed_data.shape)
```

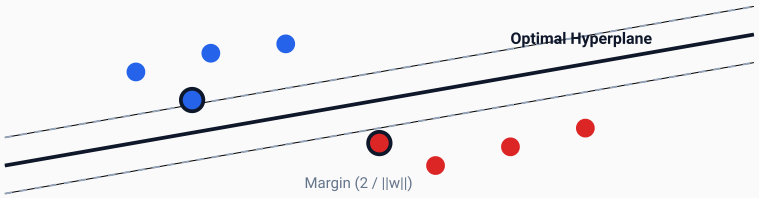
MODULE 4

Supervised Machine Learning: Classification & SVMs

4.1 Support Vector Machines (SVM)

Support Vector Machines construct an optimal decision hyper-plane that maximizes the margin between different classes. The optimization problem relies on maximizing the boundary distance *2 / ||w||*.

4.2 Concept Visualization: Decision Boundary & Support Vectors



4.3 Mathematical Formulation

Decision Function: $f(x) = \text{sign}(w^T x + b)$ | Margin Cost: $\min \frac{1}{2} ||w||^2$ s.t. $y_i(w^T x_i + b) \geq 1$

4.4 Scikit-Learn SVM Code Example

```
from sklearn.svm import SVC from sklearn.datasets import make_classification from sklearn.mod
el_selection import train_test_split X, y = make_classification(n_samples=100, n_features=2, n_redu
ndant=0, random_state=42) X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
clf = SVC(kernel='linear', C=1.0) clf.fit(X_train, y_train) print(f"Model Accuracy: {clf.score(X_te
st, y_test):.2f}") print(f"Number of Support Vectors: {clf.support_vectors_.shape[0]}")
```

MODULE 5

Unsupervised Machine Learning: K-Means Clustering

5.1 Clustering Mechanics

K-Means partitions unlabelled multidimensional datasets into *K* distinct clusters by iteratively updating centroid locations to minimize total Within-Cluster Sum of Squares (WCSS).

5.2 Concept Visualization: Centroid Optimization



5.3 Objective Equation

$$WCSS = \sum_{k=1..K} \sum_{x \in c_k} || x - \mu_k ||^2$$

5.4 Scikit-Learn K-Means Code Example

```
from sklearn.cluster import KMeans import numpy as np # Synthetic unlabeled dataset X = np.ar
ray([[1, 2], [1, 4], [1, 0], [10, 2], [10, 4], [10, 0]]) # Fit K-Means with K=2 clusters kmeans = K
Means(n_clusters=2, random_state=0).fit(X) print("Cluster Center Coordinates:\n", kmeans.cluster_ce
nters_) print("Assigned Labels:", kmeans.labels_)
```

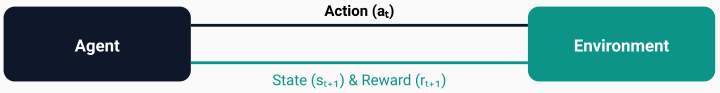
MODULE 6

Reinforcement Learning & Q-Learning

6.1 The Agent-Environment Loop

Reinforcement Learning models an agent interacting with an environment over discrete time steps. At step t, the agent observes state s_t, takes action a_t, and receives reward r_{t+1}.

6.2 Concept Visualization: MDP Loop



6.3 Bellman Equation for Q-Learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [\, r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \,]$$

6.4 Q-Learning Update Implementation

```
import numpy as np # Q-table initialization (States x Actions) Q = np.zeros((4, 2)) # 4 state
s, 2 actions alpha, gamma = 0.1, 0.9 # Learning rate & discount factor # Perform single Q-value upd
ate state, action, reward, next_state = 0, 1, 5.0, 1 best_next_action = np.argmax(Q[next_state]) #
Bellman Update Q[state, action] += alpha * (reward + gamma * Q[next_state, best_next_action] - Q[st
ate, action]) print("Updated Q-Table:\n", Q)
```

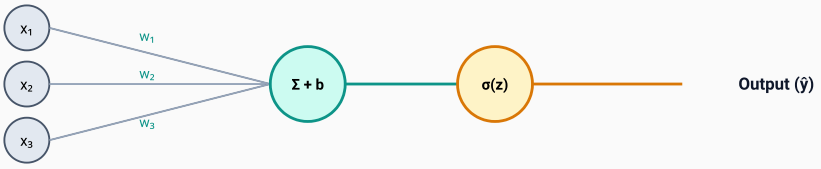
MODULE 7

Artificial Neural Networks & Backpropagation

7.1 The Mathematical Neuron

An artificial neuron computes a weighted sum of inputs plus a bias term, followed by an activation function *σ(z)* to introduce non-linearity into the network.

7.2 Concept Visualization: Neuron Architecture



7.3 Forward & Backpropagation Calculus

$$z = \sum (w_i \cdot x_i) + b \quad | \quad a = \sigma(z) = 1 / (1 + e^{-z}) \quad | \quad \partial L/\partial w = (a - y) \cdot x$$

7.4 PyTorch Multilayer Perceptron

```
import torch import torch.nn as nn class MLP(nn.Module): def __init__(self, input_dim, hidden_dim, output_dim): super(MLP, self).__init__() self.fc1 = nn.Linear(input_dim, hidden_dim) self.relu = nn.ReLU() self.fc2 = nn.Linear(hidden_dim, output_dim) def forward(self, x): out = self.fc1(x) out = self.relu(out) out = self.fc2(out) return out model = MLP(input_dim=10, hidden_dim=32, output_dim=2) print(model)
```

MODULE 8

Deep Learning Practice & Regularization

8.1 Overfitting & Regularization Techniques

Deep networks are prone to overfitting training data. Techniques such as Dropout randomly deactivate neurons during training, forcing redundant feature representations and improving generalization.

8.2 Concept Visualization: Dropout Regularization



8.3 Batch Normalization Equation

$$\hat{x} = (x - \mu_B) / \sqrt{(\sigma_B^2 + \varepsilon)} \quad | \quad y = \gamma \cdot \hat{x} + \beta$$

8.4 PyTorch Dropout & BatchNorm Block

```
import torch.nn as nn class RegularizedBlock(nn.Module): def __init__(self, dim): super().__init__() self.fc = nn.Linear(dim, dim) self.bn = nn.BatchNorm1d(dim) self.dropout = nn.Dropout(p=0.3) self.relu = nn.ReLU() def forward(self, x): return self.dropout(self.relu(self.bn(self.fc(x))))
```

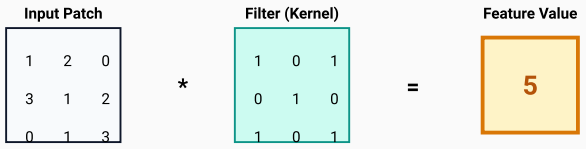
MODULE 9

CNN & Computer Vision: The Convolution Operation

9.1 Convolutional Mechanics

Convolutional Neural Networks apply parameterized filter kernels across spatial grid inputs to extract localized features such as edges, textures, and complex shapes with parameter sharing.

9.2 Concept Visualization: Numerical Convolution Example



9.3 Output Dimension Equation

$$\text{Output Size } O = \lfloor (W - K + 2P) / S \rfloor + 1$$

Where W = Input Width, K = Kernel Size, P = Padding, S = Stride.

9.4 PyTorch CNN Layer Implementation

```
import torch.nn as nn class ConvNetBlock(nn.Module): def __init__(self): super(ConvNetBlock, self).__init__() self.conv1 = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, stride=1, padding=1) self.relu = nn.ReLU() self.pool = nn.MaxPool2d(kernel_size=2, stride=2) def forward(self, x): return self.pool(self.relu(self.conv1(x)))
```

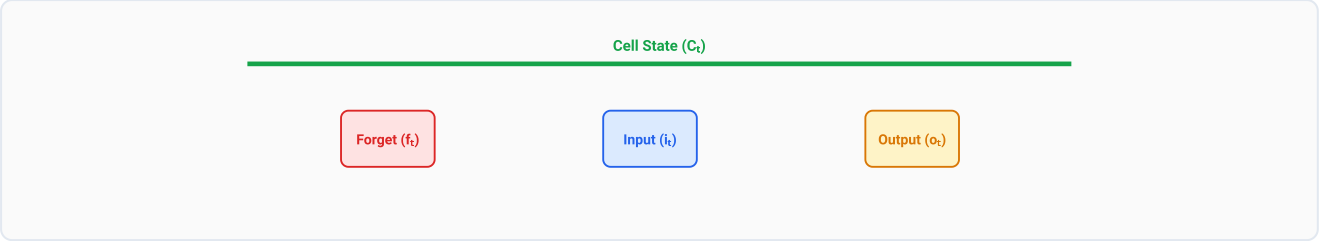
MODULE 10

RNN, LSTM & Gated Recurrent Units

10.1 Sequence Modeling & Memory Gating

Standard Recurrent Neural Networks suffer from vanishing gradients across long sequences. Long Short-Term Memory (LSTM) networks solve this using forget, input, and output gating mechanisms to regulate information flow in a persistent cell state.

10.2 Concept Visualization: LSTM Gated Cell Architecture



10.3 LSTM Gate Equations

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad | \quad i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

10.4 PyTorch LSTM Code Example

```
import torch import torch.nn as nn # LSTM Model Initialization lstm = nn.LSTM(input_size=10, hidden_size=20, num_layers=1, batch_first=True) # Input sequence: [batch_size=3, seq_len=5, input_size=10] inputs = torch.randn(3, 5, 10) out, (h_n, c_n) = lstm(inputs) print("Output Sequence Shape:", out.shape) # [3, 5, 20] print("Final Hidden State Shape:", h_n.shape) # [1, 3, 20]
```

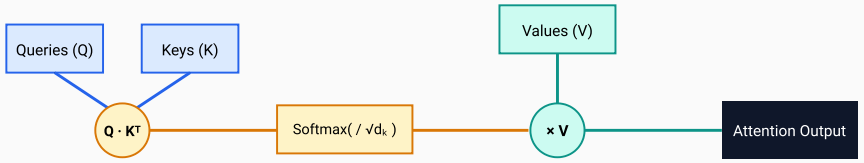
MODULE 11

Attention Mechanisms & Transformer Architecture

11.1 Scaled Dot-Product Attention

Transformers calculate dynamic contextual token relationships using Query (Q), Key (K), and Value (V) matrix representations, allowing parallelized global dependency learning.

11.2 Concept Visualization: Query-Key-Value Attention Flow



11.3 Attention Equation

$$\text{Attention}(Q, K, V) = \text{softmax}\left((Q \cdot K^T) / \sqrt{d_k} \right) \cdot V$$

11.4 PyTorch Multihead Attention Code

```
import torch import torch.nn as nn embed_dim = 64 attention = nn.MultiheadAttention(embed_dim =embed_dim, num_heads=4) # [Seq_Len=5, Batch_Size=2, Embed_Dim=64] x = torch.randn(5, 2, embed_dim) attn_output, attn_weights = attention(x, x, x) print("Output shape:", attn_output.shape)
```

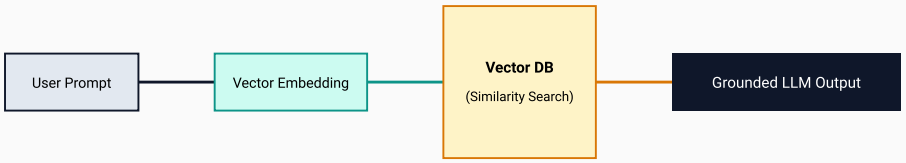
MODULE 12 & 14

Generative AI Pipelines & MLOps Deployment

12.1 Retrieval-Augmented Generation (RAG) Architecture

RAG prevents LLM hallucinations by retrieving verified external domain knowledge from vector database indices to ground generated text responses.

12.2 Concept Visualization: RAG Information Retrieval Flow



14.1 Key Glossary & Exam Review Terms

Term	Formal Definition
Overfitting	High variance state where model learns training noise instead of general patterns.
Gradient Vanishing	Attenuating loss gradients in deep networks preventing early layer weight updates.
Transformer	Attention-based neural architecture eliminating recurrence for sequence processing.
Data Drift	Statistical change in production input data distribution degrading model accuracy over time.